Linux Commands

1. Create a compressed archive using tar

Command:

tar -czvf backup.tar.gz file1.txt file2.txt
Explanation:

Creates a gzip-compressed archive named backup.tar.gz containing file1.txt and file2.txt.

Options: -c creates an archive, -z compresses it with gzip, -v displays the files being processed, and -f specifies the archive filename.

Observed result: The terminal displayed file1.txt and file2.txt, confirming that both files were added to the archive.

2. List/extract archive contents using tar

Command:

tar -xzvf backup.tar.gz
Explanation:

Extracts the files from backup.tar.gz into the current directory. The -x option extracts, -z handles gzip compression, -v displays filenames, and -f specifies the archive.

Observed result: file1.txt and file2.txt were displayed as they were extracted.

Note: Extraction can overwrite files with the same names. If prompted, check before confirming.

3. Compress a file while keeping the original

Command:

gzip -k file1.txt
Explanation:

Creates a compressed copy named file1.txt.gz. The -k option keeps the original file1.txt instead of removing it.

Observed result: gzip reported that file1.txt.gz already existed and asked whether to overwrite it. You entered y to confirm.

Note: If you do not want to overwrite an existing compressed file, answer n or use a different output filename.

4. Display compression information

Command:

gzip -l file1.txt.gz
Explanation:

Displays the compressed size, original (uncompressed) size, compression ratio, and original filename.

Observed result: compressed size 71 bytes, uncompressed size 46 bytes, ratio 6.5%, filename file1.txt. Values can vary if the file changes.

5. Print selected lines with sed

Command:

```
sed -n '1,3p' file1.txt
```

Explanation:

Prints only lines 1 through 3 of file1.txt. The -n option suppresses automatic printing, and the p command prints the selected range.

Observed result: The first three lines displayed were “Hi”, “Hello”, and “How”.

Note: This command only displays the selected lines; it does not modify the file.

6. Extract columns using awk

Create the sample data file:

```
printf '101 Ravi 85\\n102 Sita 92\\n103 Arun 78\\n' > students.txt
```

Display the first and second columns:

```
awk '{print $1, $2}' students.txt
```

Explanation:

awk processes text one line at a time. By default, it separates fields using whitespace. $1 refers to the first field and $2 to the second field.

Observed result: 101 Ravi, 102 Sita, and 103 Arun were displayed. The marks column ($3) was not printed.

7. Find text files in the home directory

Command:

```
find ~ -type f -name "*.txt"
```

Explanation:

Searches the home directory (~) and its subdirectories for regular files (-type f) whose names end in .txt.

Observed result: The command listed paths such as /home/Srini/file1.txt, names.txt, output.txt, fileA.txt, file2.txt, students.txt, and fileB.txt.

Note: The exact list depends on which text files exist in your home directory.

8. Display detailed file metadata using stat

Command:

```
stat file1.txt
```
Explanation:

Displays metadata about a file, including its size, filesystem block information, inode number, link count, file type, permissions, owner, group, and timestamps.

Observed result: file1.txt was 46 bytes and had permissions 0644 (-rw-r--r--). Its owner and group were Srini.

Timestamp meanings: Access is the last access time, Modify is the last content-modification time, Change is the last metadata-change time, and Birth is the creation time when the filesystem provides it.

9. Compare two files using diff

Create two sample files:

```
printf 'Linux\\nDocker\\nAWS\\n' > version1.txt
printf 'Linux\\nDocker\\nJenkins\\n' > version2.txt
```
Compare them:

```
diff -u version1.txt version2.txt
```
Explanation:

Compares the contents of two files. The -u option displays the differences in unified format.

Observed result: Linux and Docker were common lines. The line -AWS indicates that AWS exists in version1.txt but not version2.txt. The line +Jenkins indicates that Jenkins exists in version2.txt instead.

The --- and +++ header lines identify the old and new files. The @@ line describes the line range being compared.

10. View and modify file Access Control Lists (ACLs)

View the current ACL:

```
getfacl file1.txt
```
Add a named-user ACL entry:

```
setfacl -m u:$USER:rw file1.txt
```
Verify the ACL:

```
getfacl file1.txt
```
Explanation:

getfacl displays a file's access-control list. setfacl changes it. In the command, -m modifies/adds an entry, u identifies a user, $USER expands to the current username, and rw grants read and write permissions.

Observed result before the change: owner permissions were rw-, group permissions were r--, and other-user permissions were r--.

Observed result after the change: an additional entry user:Srini:rw- appeared, along with mask::rw-.

Important note: Srini is also the file owner in this example, so the extra named-user entry is redundant for the owner. It is still useful for demonstrating ACL syntax. The ACL mask limits the effective permissions of named users and groups.

Optional cleanup:

```
setfacl -x u:$USER file1.txt
```

Removes the named-user ACL entry. Run getfacl file1.txt afterward to verify that it is gone. The screenshot shows the ACL before cleanup, not the post-cleanup verification.

If getfacl or setfacl is unavailable, install the ACL utilities with:

```
sudo apt update && sudo apt install acl -y
```

Quick command summary

Command	Purpose
tar -czvf	Create a gzip-compressed archive
tar -xzvf	Extract a gzip-compressed archive
gzip -k	Compress while retaining the original
gzip -l	Show compression statistics
sed -n	Print selected lines
awk	Extract or process columns/fields
find	Search for files and directories
stat	Display file metadata and timestamps
diff -u	Compare file contents
getfacl / setfacl	View and modify file ACL permissions